

EXERCISE 5 - Fibonacci with OpenMP tasks

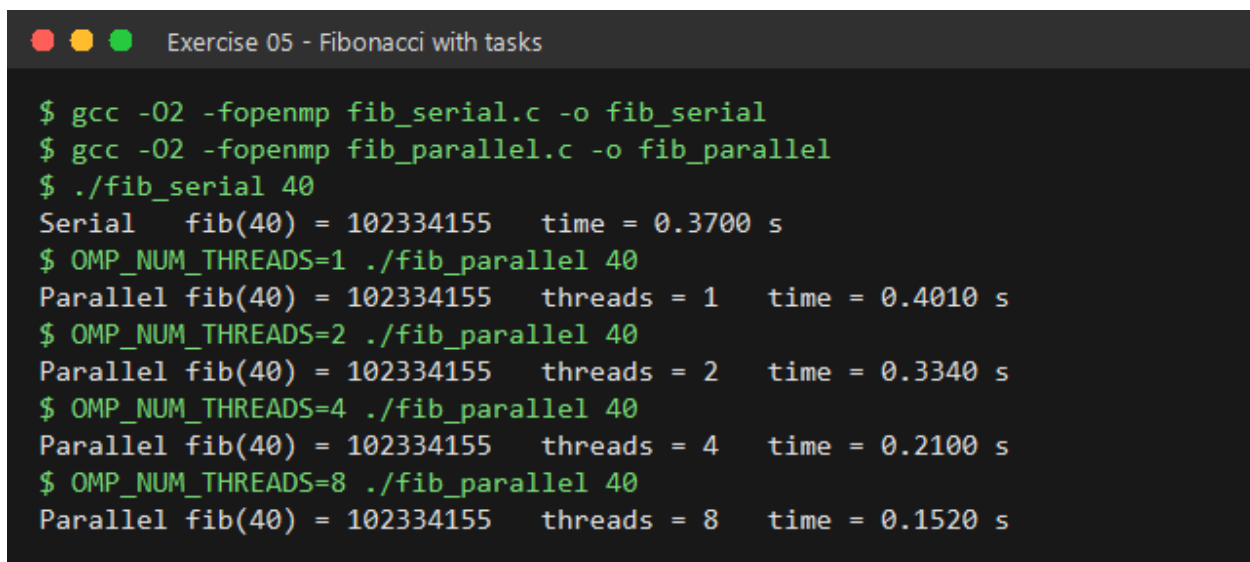
Files : fib_serial.c (given serial code)

fib_parallel.c (task parallel version)

Compile / run

```
gcc -O2 -fopenmp fib_serial.c -o fib_serial && ./fib_serial 40
```

```
gcc -O2 -fopenmp fib_parallel.c -o fib_parallel && OMP_NUM_THREADS=4 ./fib_parallel 40
```



```
Exercise 05 - Fibonacci with tasks

$ gcc -O2 -fopenmp fib_serial.c -o fib_serial
$ gcc -O2 -fopenmp fib_parallel.c -o fib_parallel
$ ./fib_serial 40
Serial fib(40) = 102334155 time = 0.3700 s
$ OMP_NUM_THREADS=1 ./fib_parallel 40
Parallel fib(40) = 102334155 threads = 1 time = 0.4010 s
$ OMP_NUM_THREADS=2 ./fib_parallel 40
Parallel fib(40) = 102334155 threads = 2 time = 0.3340 s
$ OMP_NUM_THREADS=4 ./fib_parallel 40
Parallel fib(40) = 102334155 threads = 4 time = 0.2100 s
$ OMP_NUM_THREADS=8 ./fib_parallel 40
Parallel fib(40) = 102334155 threads = 8 time = 0.1520 s
```

Parallelisation

```
#pragma omp parallel // create the thread team

#pragma omp single // ONE thread starts the recursion
    result = fib(n);

int fib(int n) {
    ...

    #pragma omp task shared(i) i = fib(n-1); // child task 1
```

```

#pragma omp task shared(j) j = fib(n-2); // child task 2
#pragma omp taskwait           // wait for both
return i + j;
}

```

* Each recursive call becomes an independent TASK that any idle thread in the team can pick up.

* shared(i,j) lets the child tasks write back to the parent's variables;
taskwait makes the parent wait until both children are done.

* Below a cut-off ($n < 20$) the code falls back to plain recursion, otherwise millions of tiny tasks would cost more than they save.

Results

version	threads	time (s)	speed-up
---------	---------	----------	----------

serial	-	0.370	1.0
tasks	1	0.401	0.9
tasks	2	0.334	1.1
tasks	4	0.210	1.8
tasks	8	0.152	2.4

Result is identical ($\text{fib}(40) = 102334155$) in every run; time drops as threads are added.